

Python Data Science

Quick Reference Cheatsheet

NumPy · Pandas · Matplotlib · Seaborn · EDA · Preprocessing

1. NumPy — Numerical Computing

Import & Array Creation

```
import numpy as np

a = np.array([1, 2, 3]) # 1-D array
b = np.array([[1,2],[3,4]]) # 2-D array
np.zeros((3, 4)) # 3x4 zeros
np.ones((2, 3)) # 2x3 ones
np.eye(4) # 4x4 identity
np.arange(0, 10, 2) # [0,2,4,6,8]
np.linspace(0, 1, 5) # 5 evenly spaced values
np.random.rand(3, 3) # uniform [0,1)
np.random.randn(3, 3) # standard normal
np.random.randint(0, 10, (3, 3)) # random ints
```

Shape & Indexing

Shape / Reshape

```
a.shape
a.ndim
a.dtype
a.size
a.reshape(2, 6) # reshape
a.flatten() # 1-D copy
a.T # transpose
np.squeeze(a) # remove size-1 dims
np.expand_dims(a,0) # add dim at axis 0
```

Indexing

```
a[0] # first element
a[-1] # last element
a[1:4] # slice
a[::2] # every 2nd
a[a > 5] # boolean mask
np.where(a>0, a, 0) # conditional
np.argmax(a) # index of max
np.argmin(a) # index of min
np.argsort(a) # sorted indices
```

Math Operations

Stats & Reductions

```
np.sum(a, axis=0) # col sums
np.mean(a) # mean
np.std(a) # std dev
np.var(a) # variance
np.median(a) # median
np.min(a) / np.max(a)
np.cumsum(a) # cumulative sum
np.diff(a) # differences
np.abs(a) # absolute value
```

Math Functions

```
np.sqrt(a) # square root
np.log(a) # natural log
np.log2(a) # log base 2
np.exp(a) # e^x
np.dot(a, b) # dot product
a @ b # matrix multiply
np.linalg.inv(a) # inverse
np.linalg.det(a) # determinant
np.linalg.eig(a) # eigenvalues
```

Useful Tricks

```
np.concatenate([a, b], axis=0) # stack vertically
np.stack([a, b], axis=1) # new axis stack
np.vstack([a, b]) # vertical stack
np.hstack([a, b]) # horizontal stack
np.split(a, 3) # split into 3 equal parts
np.unique(a, return_counts=True) # unique values + counts
np.clip(a, 0, 1) # clip values to [0,1]
np.nan_to_num(a) # replace NaN with 0
np.save('arr.npy', a) # save; np.load() to load
```

2. Pandas — Data Manipulation

Import & Creation

```
import pandas as pd

df = pd.DataFrame({'A':[1,2,3], 'B':[4,5,6]})
df = pd.read_csv('file.csv', sep=',', encoding='utf-8')
df = pd.read_excel('file.xlsx', sheet_name='Sheet1')
df = pd.read_json('file.json')
df = pd.read_sql(query, connection) # from SQL
df.to_csv('out.csv', index=False) # save CSV
df.to_excel('out.xlsx', index=False) # save Excel
```

Inspection

Basic Inspection

```
df.head(n) # first n rows
df.tail(n) # last n rows
df.shape # (rows, cols)
df.dtypes # column types
df.columns # column names
df.index # row index
df.info() # overview
df.describe() # stats summary
```

Advanced Inspection

```
df.nunique() # unique counts
df.value_counts() # freq table
df.isnull().sum() # missing per col
df.duplicated().sum() # duplicate rows
df.memory_usage() # memory per col
df.corr() # correlation matrix
df.skew() # skewness
df.kurtosis() # kurtosis
```

Selection & Filtering

```
df['col'] # Series
df[['col1', 'col2']] # DataFrame
```

```

df.loc[0, 'col'] # label-based
df.iloc[0, 1] # position-based
df.loc[df['A'] > 2] # filter rows
df[(df.A>1) & (df.B<5)] # multiple conditions
df.query('A > 1 and B < 5') # query string
df.filter(regex='^col') # regex col filter
df.nlargest(5, 'A') # top 5 by col A

```

Transformation

Column Ops

```

df['new'] = df['A'] * 2
df.assign(new=lambda x: x.A*2)
df.rename(columns={'A':'a'})
df.drop('col', axis=1)
df.drop_duplicates()
df.reset_index(drop=True)
df.set_index('col')
df.sort_values('A', ascending=False)

```

Apply & Map

```

df['A'].apply(func) # apply fn
df.applymap(func) # element-wise
df.map({'old':'new'}) # map values
pd.get_dummies(df['cat']) # one-hot
df.astype({'A': float}) # cast type
df.clip(lower=0, upper=10)
df.replace(-999, np.nan) # replace val
df.T # transpose

```

GroupBy & Aggregation

```

df.groupby('cat')['val'].mean() # group mean
df.groupby('cat').agg({'A':'sum', 'B':'mean'}) # multi-agg
df.groupby('cat').agg(['min', 'max', 'std']) # multi-stat
df.groupby('cat').transform('mean') # broadcast back
df.pivot_table(values='val', index='r', columns='c', aggfunc='sum')
df.pivot(index='date', columns='cat', values='val')
df.melt(id_vars=['id'], value_vars=['A','B']) # wide -> long

```

Merge & Concat

```

pd.merge(df1, df2, on='key', how='inner') # SQL-style join
pd.merge(df1, df2, left_on='a', right_on='b', how='left')
pd.concat([df1, df2], axis=0, ignore_index=True) # stack rows
pd.concat([df1, df2], axis=1) # stack cols

```

Date/Time

```

df['date'] = pd.to_datetime(df['date']) # parse dates
df['year'] = df['date'].dt.year
df['month'] = df['date'].dt.month
df['dow'] = df['date'].dt.day_name() # Monday etc.
df.set_index('date').resample('M').sum() # monthly resample
pd.date_range('2024-01-01', periods=12, freq='MS') # date range

```

3. Matplotlib — Plotting

Basic Setup

```

import matplotlib.pyplot as plt
import matplotlib as mpl

fig, ax = plt.subplots(figsize=(10, 6)) # recommended OO style

```

```
fig, axes = plt.subplots(2, 3, figsize=(14, 8)) # grid of plots
plt.style.use('seaborn-v0_8-whitegrid') # set style
mpl.rcParams['figure.dpi'] = 150 # increase resolution
```

Common Plot Types

Line / Bar / Scatter

```
ax.plot(x, y, color='blue', lw=2) # line
ax.scatter(x, y, c=z, cmap='viridis') # scatter
ax.bar(cats, vals, color='steelblue') # bar
ax.barh(cats, vals) # horizontal bar
ax.hist(data, bins=30, edgecolor='k') # histogram
ax.boxplot(data) # box plot
ax.violinplot(data) # violin
```

Others

```
ax.pie(sizes, labels=labels, autopct='%1.1f%%')
ax.fill_between(x, y1, y2, alpha=0.3) # shaded area
ax.errorbar(x, y, yerr=err, fmt='o') # error bars
ax.imshow(matrix, cmap='hot') # heatmap/image
ax.contourf(X, Y, Z, levels=20) # contour
ax.step(x, y) # step plot
ax.stem(x, y) # stem plot
```

Customization

```
ax.set_title('Title', fontsize=16, fontweight='bold')
ax.set_xlabel('X Label', fontsize=12)
ax.set_ylabel('Y Label', fontsize=12)
ax.set_xlim(0, 100) / ax.set_ylim(-1, 1)
ax.set_xticks(range(0,100,10)) # manual ticks
ax.xaxis.set_tick_params(rotation=45)
ax.legend(loc='upper right', fontsize=10)
ax.grid(True, linestyle='--', alpha=0.7)
ax.spines['top'].set_visible(False) # remove spines
ax.annotate('peak', xy=(x,y), xytext=(x+1,y+5),
arrowprops=dict(arrowstyle='->'))
plt.tight_layout() # prevent overlap
plt.savefig('plot.png', dpi=200, bbox_inches='tight')
```

Advanced Tricks

```
# Twin axes
ax2 = ax.twinx(); ax2.plot(x, y2, color='red')

# Colorbar
sc = ax.scatter(x, y, c=z); plt.colorbar(sc, ax=ax)

# Log scale
ax.set_yscale('log')

# Subplot spacing
fig.subplots_adjust(hspace=0.4, wspace=0.3)

# Secondary x-axis
secax = ax.secondary_xaxis('top', functions=(f, finv))
```

4. Seaborn — Statistical Visualization

Import & Themes

```
import seaborn as sns
```

```
sns.set_theme(style='whitegrid', palette='muted', font_scale=1.1)
sns.set_style('darkgrid') # whitegrid | dark | white | ticks
sns.set_palette('husl') # deep | muted | pastel | husl | Set2
sns.reset_defaults() # reset all settings
```

Categorical Plots

Plot Types	Hue & Order
<code>sns.barplot(data=df, x='cat', y='val')</code>	<code># hue parameter adds color grouping</code>
<code>sns.countplot(data=df, x='cat')</code>	<code>sns.boxplot(data=df, x='cat',</code>
<code>sns.boxplot(data=df, x='cat', y='val')</code>	<code>y='val', hue='grp')</code>
<code>sns.violinplot(data=df, x='cat', y='val',</code>	<code># order parameter</code>
<code>split=True)</code>	<code>sns.barplot(data=df, x='cat',</code>
<code>sns.stripplot(data=df, x='cat', y='val')</code>	<code>y='val',</code>
<code>sns.swarmplot(data=df, x='cat', y='val')</code>	<code>order=df.cat.value_counts()</code>
	<code>.index)</code>

Distribution & Relational Plots

```
sns.histplot(data=df, x='val', bins=30, kde=True, hue='cat')
sns.kdeplot(data=df, x='val', fill=True, hue='cat', alpha=0.5)
sns.ecdfplot(data=df, x='val') # cumulative dist
sns.rugplot(data=df, x='val') # rug plot
sns.scatterplot(data=df, x='A', y='B', hue='cat', size='val')
sns.lineplot(data=df, x='date', y='val', hue='cat', ci=95)
sns.regplot(data=df, x='A', y='B', scatter_kws={'alpha':0.4})
sns.residplot(data=df, x='A', y='B') # residuals
```

Matrix & Multi-plot

```
sns.heatmap(df.corr(), annot=True, fmt='.2f', cmap='coolwarm',
vmin=-1, vmax=1, square=True, linewidths=0.5)

sns.clustermap(df.corr(), figsize=(10, 10)) # clustered heatmap

sns.pairplot(df, hue='target', diag_kind='kde', plot_kws={'alpha':0.5})

g = sns.FacetGrid(df, col='cat', row='grp', height=4)
g.map(sns.histplot, 'val')

sns.jointplot(data=df, x='A', y='B', kind='hex') # hex|kde|reg|resid
```

5. EDA — Exploratory Data Analysis Steps

Step 1 — Load & First Look

```
df = pd.read_csv('data.csv')
print(df.shape) # dimensions
df.head() # first 5 rows
df.info() # dtypes + non-null counts
df.describe(include='all') # stats for all columns
```

Step 2 — Missing Values

```
df.isnull().sum() # missing per column
df.isnull().mean() * 100 # % missing
```

```
sns.heatmap(df.isnull(), cbar=False, yticklabels=False) # visual
df[df.isnull().any(axis=1)] # rows with any NaN
df.dropna(thresh=len(df)*0.5, axis=1) # drop cols >50% NaN
```

Step 3 — Univariate Analysis

```
# Numerical
df['col'].describe() # 5-number summary
sns.histplot(df['col'], kde=True) # distribution
sns.boxplot(y=df['col']) # outlier detection

# Categorical
df['cat'].value_counts() # frequencies
df['cat'].value_counts(normalize=True) # proportions
sns.countplot(data=df, x='cat', order=df['cat'].value_counts().index)
```

Step 4 — Bivariate & Multivariate Analysis

```
# Num vs Num
df[['A','B']].corr() # Pearson correlation
sns.scatterplot(data=df, x='A', y='B') # scatter
sns.regplot(data=df, x='A', y='B') # with trendline

# Cat vs Num
df.groupby('cat')['num'].describe()
sns.boxplot(data=df, x='cat', y='num')

# All vs All
sns.pairplot(df, hue='target')
sns.heatmap(df.corr(), annot=True, cmap='coolwarm')
```

Step 5 — Outlier Detection

```
# IQR Method
Q1 = df['col'].quantile(0.25)
Q3 = df['col'].quantile(0.75)
IQR = Q3 - Q1
outliers = df[(df['col'] < Q1-1.5*IQR) | (df['col'] > Q3+1.5*IQR)]

# Z-Score Method
from scipy import stats
z = np.abs(stats.zscore(df.select_dtypes(include=np.number)))
outliers = df[(z > 3).any(axis=1)]
```

Step 6 — Feature Relationships & Insights

```
df.groupby('target').mean() # mean by class
pd.crosstab(df['A'], df['B'], normalize='index') # pivot freq
df.corr()['target'].sort_values() # correlation with target
df.skew().sort_values() # most skewed features
```

6. Preprocessing — Data Cleaning & Feature Engineering

Handle Missing Values

```
# Drop
df.dropna() # drop any NaN row
```

```

df.dropna(subset=['col1','col2']) # specific cols only
df.dropna(thresh=3) # keep rows with >=3 non-null

# Imputation
df['num'].fillna(df['num'].mean()) # mean imputation
df['num'].fillna(df['num'].median()) # median (robust)
df['cat'].fillna(df['cat'].mode()[0]) # mode for categorical
df.ffill() / df.bfill() # forward / backward fill

# sklearn
from sklearn.impute import SimpleImputer, KNNImputer
imp = SimpleImputer(strategy='median')
df[num_cols] = imp.fit_transform(df[num_cols])
knn_imp = KNNImputer(n_neighbors=5)

```

Encoding Categorical Variables

```

# Label Encoding
from sklearn.preprocessing import LabelEncoder
le = LabelEncoder()
df['enc'] = le.fit_transform(df['cat'])

# One-Hot Encoding
df = pd.get_dummies(df, columns=['cat'], drop_first=True, dtype=int)

# Ordinal Encoding
from sklearn.preprocessing import OrdinalEncoder
oe = OrdinalEncoder(categories=[['Low', 'Mid', 'High']])
df[['ord']] = oe.fit_transform(df[['ord_col']])

# Target / Mean Encoding
means = df.groupby('cat')['target'].mean()
df['cat_enc'] = df['cat'].map(means)

```

Scaling & Normalization

```

from sklearn.preprocessing import (StandardScaler, MinMaxScaler,
RobustScaler, Normalizer)

scaler = StandardScaler() # mean=0, std=1 – Gaussian data
scaler = MinMaxScaler() # [0,1] – bounded data
scaler = RobustScaler() # IQR-based – with outliers
scaler = Normalizer(norm='l2') # unit norm per sample

X_train_sc = scaler.fit_transform(X_train) # fit only on train!
X_test_sc = scaler.transform(X_test) # never re-fit on test

```

Outlier Handling

```

# Capping / Winsorization
lo, hi = df['col'].quantile([0.01, 0.99])
df['col'] = df['col'].clip(lo, hi)

# Log transform (right-skewed)
df['col_log'] = np.log1p(df['col']) # log(x+1) safe for 0s

# Box-Cox (requires positive values)
from scipy.stats import boxcox

```

```
df['col_bc'], lam = boxcox(df['col'])
```

Feature Engineering

```
# Binning
df['age_band'] = pd.cut(df['age'], bins=[0,18,35,60,100],
labels=['teen','adult','mid','senior'])
df['qcut'] = pd.qcut(df['val'], q=4, labels=['Q1','Q2','Q3','Q4'])

# Polynomial Features
from sklearn.preprocessing import PolynomialFeatures
poly = PolynomialFeatures(degree=2, include_bias=False)
X_poly = poly.fit_transform(X)

# Interaction term manually
df['A_x_B'] = df['A'] * df['B']
```

Feature Selection

```
# Variance Threshold (remove low-variance features)
from sklearn.feature_selection import VarianceThreshold
sel = VarianceThreshold(threshold=0.01)
X_sel = sel.fit_transform(X)

# Correlation filter
corr = df.corr().abs()
upper = corr.where(np.triu(np.ones(corr.shape), k=1).astype(bool))
drop_cols = [c for c in upper.columns if any(upper[c] > 0.95)]
df.drop(drop_cols, axis=1, inplace=True)

# SelectKBest
from sklearn.feature_selection import SelectKBest, f_classif
selector = SelectKBest(f_classif, k=10)
X_new = selector.fit_transform(X, y)
selected = X.columns[selector.get_support()]
```

Train/Test Split & Pipeline

```
from sklearn.model_selection import train_test_split
from sklearn.pipeline import Pipeline
from sklearn.compose import ColumnTransformer

X_train, X_test, y_train, y_test = train_test_split(
X, y, test_size=0.2, random_state=42, stratify=y)

# Robust pipeline
num_pipe = Pipeline([('imp', SimpleImputer(strategy='median')),
('sc', StandardScaler())])
cat_pipe = Pipeline([('imp', SimpleImputer(strategy='most_frequent')),
('ohe', OneHotEncoder(handle_unknown='ignore'))])
pre = ColumnTransformer([('num', num_pipe, num_cols),
('cat', cat_pipe, cat_cols)])
final_pipe = Pipeline([('pre', pre), ('model', model)])
final_pipe.fit(X_train, y_train)
```

